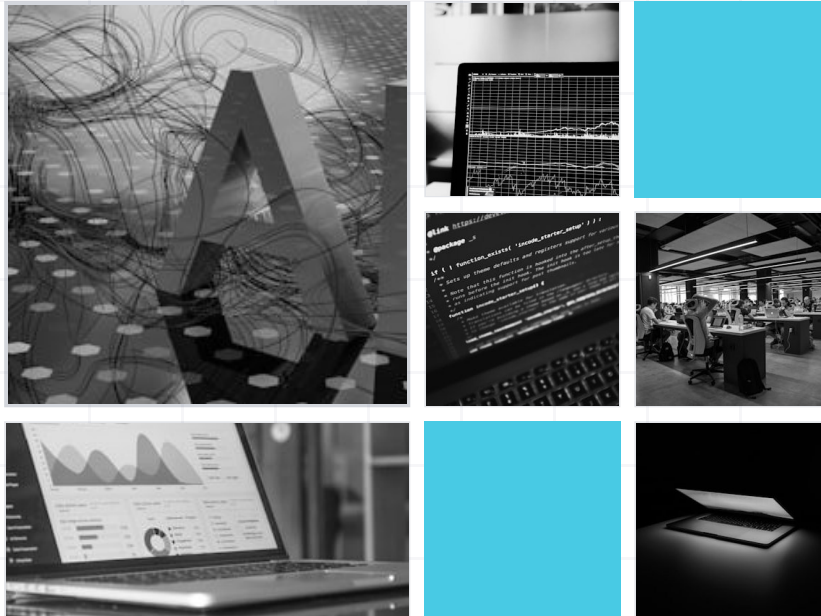




DOKUMENTASI TEKNIS LENGKAP

NORA — Network **Oracle** for Reliable Answers

Platform AI Research Engine multi-topik untuk standar telekomunikasi — jawaban terverifikasi, tergrounded pada spesifikasi resmi, dengan confidence score dan sumber per-section.

Disiapkan oleh **Tim ArahKarya** · Kolaborasi PT Arah Karya Sinergi x NOZ

2026

[Overview](#) · [Tutorial](#) · [Glosarium](#)

AKS · v1.0

Daftar Isi

Bagian I **Penjelasan Teknis**

Arsitektur sistem, alur kerja RAG, embedding, knowledge base, keamanan & deployment.

Bagian II **Tutorial Operasional**

Setup dari nol, ingest knowledge base, query, ganti embedding backend, operasional harian & troubleshooting.

Bagian III **Glosarium**

Definisi 60+ istilah: RAG & AI, komponen NORA, telco/3GPP, infrastruktur & operasional.

BAGIAN I

Penjelasan Teknis

Arsitektur sistem, alur kerja RAG, embedding, knowledge base, keamanan & deployment.

NORA — Network Oracle for Reliable Answers

Dokumen Teknis Lengkap

Atribut	Detail
Dokumen	NORA Technical Overview
Versi	1.0
Tanggal	Juni 2026
Disiapkan oleh	Tim ArahKarya
Kolaborasi	PT Arah Karya Sinergi x NOZ (Tri Sumarno)
Status	Final — siap render PDF

Ringkasan Eksekutif

NORA (Network Oracle for Reliable Answers) adalah platform SaaS *AI Research Engine* multi-topik yang dirancang khusus untuk domain standar telekomunikasi. NORA menjawab pertanyaan teknis engineer telco dengan jawaban yang **tergrounded pada spec resmi** — bukan dari ingatan model — sehingga risiko halusinasi ditekan hingga nol.

Setiap jawaban NORA dilengkapi:

- **Confidence score** (0–1) sebagai ukuran keyakinan sistem.
- **Referensi sumber** per nomor section spec (mis. §4.7.3.1).
- **Verdict verifikasi** (VALID / PARTIAL / INVALID) dari model kedua yang independen.

NORA dibangun sebagai **kolaborasi teknis NOZ (Tri Sumarno) x PT Arah Karya Sinergi**, dengan arsitektur multi-tenant yang siap scale ke banyak pengguna paralel. Platform saat ini berjalan secara **live** di nora.arahkarya.com.

1. Apa Itu NORA & Masalah yang Dipecahkan

1.1 Masalah

Engineer telekomunikasi menghadapi tantangan nyata saat harus merujuk standar teknis:

Masalah	Dampak
Spec 3GPP ratusan halaman (TS 24.008 ~600+ hal., 257 versi rilis)	Waktu riset lama, melelahkan
Versi spec berubah tiap rilis (R98 s.d. R18)	Rawan salah interpretasi antar-versi
LLM umum menjawab dari <i>parametric memory</i>	Halusinasi pada detail teknis (nomor section, IE, prosedur) — tidak dapat diterima untuk domain regulasi
Tidak ada jejak audit	Jawaban hilang, tidak bisa ditelusuri

1.2 Solusi NORA

NORA mengingesti **spec resmi** ke vector database, lalu menjawab melalui pipeline **Retrieval-Augmented Generation (RAG) + dual-model + validation layer**:

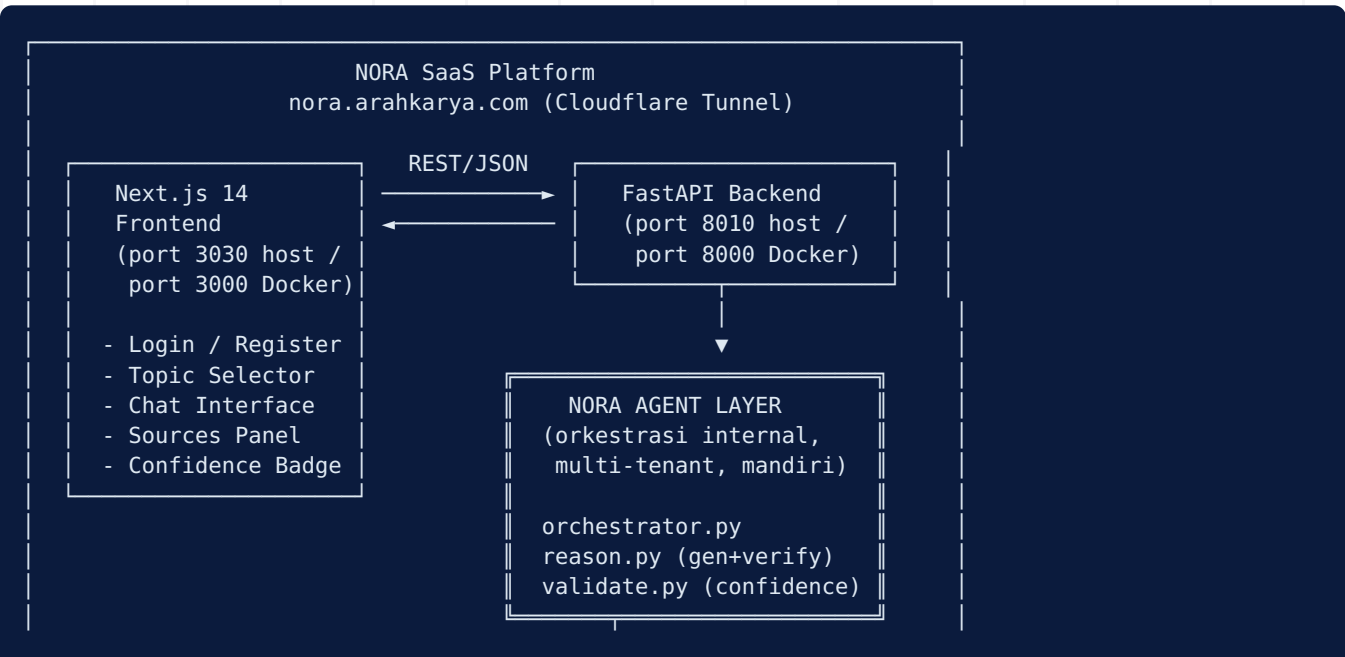
1. Sumber jawaban selalu spec asli yang diindeks — bukan ingatan model.
2. Model **Generator** menyusun jawaban dari konteks yang di-retrieve.
3. Model **Verifier** independen memverifikasi setiap klaim sebelum jawaban dikirim ke user.
4. Jika tidak ada konteks relevan → sistem menjawab "*tidak ditemukan dalam spec*" dan menolak mengarang.

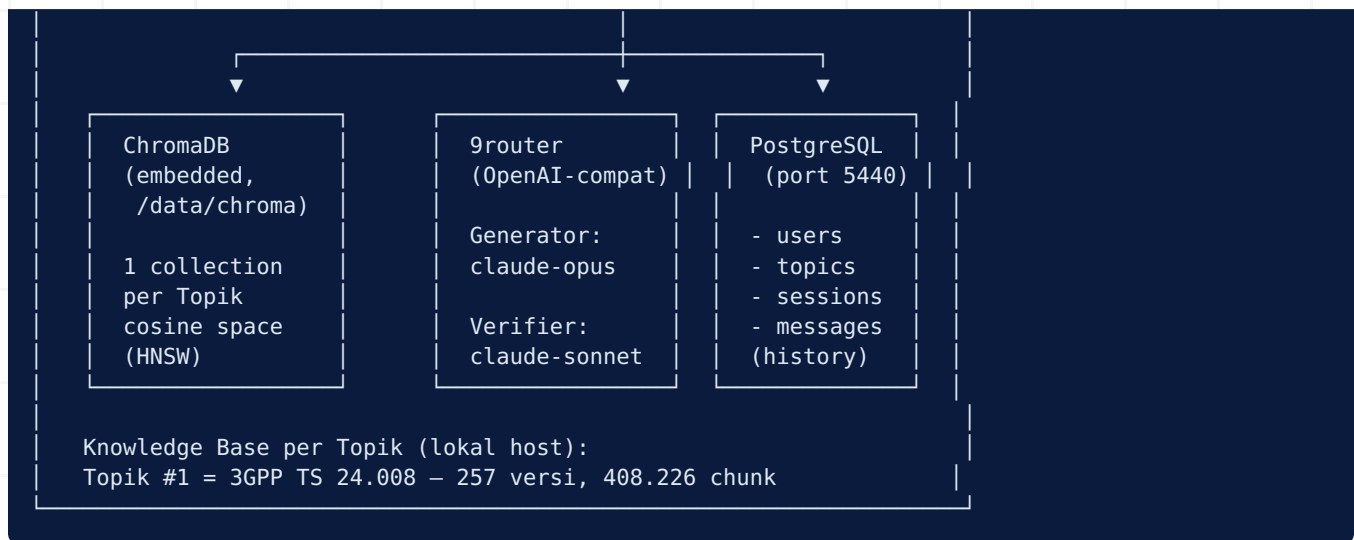
1.3 Proposisi Nilai Inti

#	Nilai	Penjelasan
1	Anti-halusinasi	Jawaban wajib bersumber dari spec resmi yang terindeks
2	Self-validating	Model kedua (Verifier) mengecek jawaban model pertama (Generator)
3	Auditable	Setiap jawaban punya confidence score + sumber yang dapat ditelusuri
4	Multi-topik & extensible	Satu engine RAG, banyak knowledge base; topik baru tidak perlu ubah kode engine
5	Engine-agnostic	Cloud (9router) atau lokal (Ollama), satu arsitektur

2. Arsitektur Sistem

2.1 Diagram Layer





2.2 Penjelasan Tiap Komponen

Frontend — Next.js 14

- Framework React dengan Server-Side Rendering (SSR).
- Berjalan di port `3030` (host) / `3000` (Docker container).
- Fitur utama: halaman login/register, topic selector, chat interface, sources panel expandable, confidence badge.
- Semua request `/api/*` di-proxy ke backend internal melalui Next.js rewrites — satu origin, cookie aman, tanpa CORS issue.
- File: `frontend/`

Backend — FastAPI

- API layer Python async, berjalan di port `8010` (host) / `8000` (Docker container).
- Bertanggung jawab atas: autentikasi, routing request, persistensi ke PostgreSQL, dan pemanggilan NORA Agent Layer sebagai service internal.
- Routes:
 - `POST /api/auth/register` — registrasi user baru
 - `POST /api/auth/login` — login, set JWT cookie
 - `GET /api/auth/me` — info user aktif
 - `GET /api/topics` — daftar topik tersedia
 - `POST /api/query` — kirim pertanyaan, dapatkan jawaban RAG
 - `GET /api/history` — riwayat chat per-user
- Swagger/ReDoc dimatikan di produksi (`NORA_ENABLE_DOCS=false` default) untuk mengurangi attack surface.
- File: `backend/nora/main.py` , `backend/nora/api/routes_*.py`

NORA Agent Layer — Orkestrasi Internal

- Engine orkestrasi mandiri milik NORA, ditulis sebagai kode Python murni (bukan runtime generik eksternal).
- Dipanggil backend sebagai library/service internal — stateless per-request, state durable di PostgreSQL.

- Menjalankan loop: **retrieve → generate → verify → validate → loop jika INVALID**.
- Multi-tenant: setiap request terisolasi per-user; memory dan history tersimpan di Postgres, bukan memori global.
- File: `backend/nora/pipeline/orchestrator.py`, `backend/nora/pipeline/reason.py`, `backend/nora/pipeline/validate.py`

ChromaDB — Vector Store

- Embedded (tidak berjalan sebagai service terpisah), disimpan di `backend/data/chroma` (di-mount ke `/app/data/chroma` di container).
- Satu collection per Topik (isolasi data penuh antar-Topik).
- Menggunakan **cosine similarity** dengan indeks HNSW.
- Embedding disuplai oleh sistem (bukan ChromaDB yang generate) — konsisten antara ingest dan query.
- File: `backend/nora/rag/store.py`

PostgreSQL — State & Auth

- Berjalan di port `5440` (host) / `5432` (internal Docker network).
- Menyimpan: tabel `users`, `topics`, `chat_sessions`, `messages` (dengan `sources_json` dan `confidence`).
- Dijaga dengan healthcheck sebelum backend start.
- File: `backend/nora/db/models.py`

9router — LLM & Embedding Gateway

- OpenAI-compatible API gateway yang berjalan di `localhost:20128` (host RPi5, diakses backend via `host.docker.internal`).
- Melayani dua model sekaligus: Generator (Opus) dan Verifier (Sonnet).
- Juga melayani endpoint embedding Gemini (`gemini-embedding-001`).

3. Alur Kerja RAG End-to-End

3.1 Diagram Alur

```

User kirim query
  |
  v
[1] FastAPI: autentikasi JWT → validasi input (max 4000 char, top_k ≤ 20)
  |
  v
[2] NORA Agent Layer: panggil answer_query(query, top_k, version_filter)
  |
  v
[3] RAG Store: embed query → ChromaDB cosine query → top-K chunk relevan
  |
  |   Jika chunk kosong (tidak ada konteks):
  |   └─ return {answer: "tidak ditemukan", confidence: 0.0, flag: "NO CONTEXT"}
  |
  v
[4] Generator (Claude Opus via 9router):
    prompt = [System: jawab HANYA dari konteks] + [Context: top-K chunk] + [Query]
    → jawaban awal
  
```

```

↓
[5] Verifier (Claude Sonnet via 9router):
    prompt = [Context] + [Jawaban awal]
    → output JSON: {verdict: VALID|PARTIAL|INVALID, reason, unsupported_claims}
    ↓
    Jika INVALID (maks 2x loop):
        ↳ [4'] Generator re-generate dengan hint dari Verifier
        ↳ [5'] Verifier re-check → lanjut ke [6]
    ↓
[6] Validate: hitung confidence score
    score = (0.6 × verdict_weight) + (0.4 × avg_similarity) - penalty_unsupported
    flag = "LOW CONFIDENCE" jika score < 0.7
    ↓
[7] Susun output:
    {answer, confidence, flag, verifier_verdict, verifier_reason,
     regeneration_count, sources[], query_id}
    ↓
[8] FastAPI: persist ke PostgreSQL (ChatSession + Message user + Message assistant)
    ↓
[9] Response JSON → Next.js → tampil di UI (jawaban + badge confidence + sumber)

```

3.2 Langkah Bernomor (Deskriptif)

Langkah	Proses	Komponen
1	User mengirim pertanyaan melalui chat UI	Frontend → FastAPI
2	Backend memvalidasi JWT dan input, lalu memanggil Agent Layer	<code>routes_query.py</code>
3	Agent Layer meng-embed query → mencari top-K chunk paling relevan dari ChromaDB (cosine similarity)	<code>orchestrator.py</code> , <code>store.py</code>
4	Jika tidak ada chunk relevan, sistem langsung mengembalikan "tidak ditemukan" tanpa memanggil LLM	<code>orchestrator.py</code>
5	Generator (Claude Opus) menyusun jawaban berdasarkan konteks chunk yang di-retrieve	<code>reason.py</code> — <code>generate()</code>
6	Verifier (Claude Sonnet) menilai apakah semua klaim jawaban didukung konteks → VALID / PARTIAL / INVALID	<code>reason.py</code> — <code>verify()</code>
7	Jika INVALID, Generator melakukan regenerasi dengan hint dari Verifier (maks. 2 kali)	<code>reason.py</code> — <code>generate_verified()</code>
8	Confidence score dihitung dari: bobot verdict + rata-rata similarity + penalti klaim tidak bersumber	<code>validate.py</code> — <code>confidence()</code>
9	Output lengkap dikembalikan ke user; percakapan disimpan ke PostgreSQL	<code>orchestrator.py</code> , <code>routes_query.py</code>

3.3 Fallback Chain LLM

NORA mengimplementasi fallback otomatis jika model utama overload atau timeout:

Peran	Model Utama	Fallback 1	Fallback 2
Generator	cc/claude-opus-4-8	cc/claude-sonnet-4-6	ag/gemini-3-flash
Verifier	cc/claude-sonnet-4-6	ag/gemini-3-flash	—

Error transient yang memicu fallback: HTTP 429, 500, 502, 503, 529, timeout, overload.

4. Embedding & Vector Store

4.1 Multi-Backend Embedding

NORA mendukung tiga backend embedding yang dapat dipilih via environment variable `NORA_EMBED_BACKEND`. Backend yang dipakai saat ingest **harus sama** dengan saat query — dimensi vektor harus konsisten.

```
NORA_EMBED_BACKEND=9router # default – Gemini cloud via 9router
NORA_EMBED_BACKEND=ollama # Ollama lokal (offline)
NORA_EMBED_BACKEND=local # fastembed ONNX CPU (paling ringan)
```

4.2 Tabel Perbandingan Backend Embedding

Backend	Provider	Model Default	Dimensi Vektor	Mode	Keterangan
9router (default)	Google / 9router	gemini/gemini-embedding-001	3072	Cloud	Kualitas terbaik; RAM RPi5 aman (komputasi di cloud)
ollama	Ollama (lokal)	nomic-embed-text	768	Lokal offline	Privasi penuh; mxbai-embed-large = 1024 dim
local	fastembed ONNX	BAAI/bge-small-en-v1.5	384	Lokal offline CPU	Paling ringan; cocok resource terbatas

Peringatan: Ganti backend embedding = wajib re-ingest seluruh knowledge base. Index dan query harus memakai backend + model yang sama.

4.3 ChromaDB Vector Store

- **Tipe:** Embedded (tidak ada service terpisah), persisten di disk.
- **Path:** `backend/data/chroma` (host) → `/app/data/chroma` (container).
- **Konfigurasi env:** `NORA_CHROMA_PATH`, `NORA_COLLECTION`.
- **Similarity metric:** Cosine (HNSW space).
- **Struktur collection:** 1 collection per Topik, nama diambil dari field `collection_name` di tabel `topics`.
- **Topik #1:** collection `ts24008`, berisi seluruh chunk 3GPP TS 24.008.

4.4 Proses Ingest ke ChromaDB

```
# backend/nora/rag/store.py – add_chunks()
col.upsert(ids=..., embeddings=embed_texts(texts), documents=texts, metadatas=metas)
```

- Batch size 32 teks per call ke embedding API (aman di bawah limit Gemini).
- Deduplication: jika `chroma_id` sama (section muncul dua kali), suffix counter ditambahkan.
- `upsert` — aman untuk re-ingest (tidak duplikat).

5. Knowledge Base & Chunking

5.1 Knowledge Base Topik #1: 3GPP TS 24.008

3GPP TS 24.008 adalah spesifikasi *Mobile Radio Interface Layer 3* yang mendefinisikan protokol inti jaringan 2G/3G/4G: **Mobility Management (MM)**, **GPRS Mobility Management (GMM)**, **Call Control (CC)**, dan **Session Management (SM)**.

Item	Detail
Spec	TS 24.008 — Mobile radio interface Layer 3; Core network protocols; Stage 3
Cakupan	MM / GMM / CC / SM
Jumlah versi	257 rilis (R98 v3.0.0 → R18 j-series, tahun 1999–2026)
Format mentah	<code>.doc</code> / <code>.docx</code> (per zip dari 3GPP FTP) — total ~778 MB
Format diolah	<code>.txt</code> via LibreOffice headless — ~229 MB
Lokasi host	<code>~/data/3gpp/24008/txt</code>
Total chunk terindeks	~ 408.226 chunk (rata-rata ~1.051 chunk/versi)
Collection ChromaDB	<code>ts24008</code>

5.2 Section-Aware Chunking

File `.txt` hasil konversi LibreOffice dipecah berdasarkan **nomor section 3GPP** (bukan ukuran fixed), sesuai PRD FR-ING-002.

Regex pendeteksi heading section:

```
SECTION_RE = re.compile(
    r'^\s*(?P<num>\d{1,2}(?:\.\d{1,3}){0,5})\s*(?:\t |(?P<title>[A-Z][^\n]{2,120}))\s*$'
)
```

Contoh heading yang terdeteksi: `4.7.3 GPRS attach procedure`

Parameter chunking:

Parameter	Nilai	Keterangan
<code>MAX_CHARS</code>	3200 char (~800 token)	Batas maksimum satu chunk
<code>OVERLAP_CHARS</code>	200 char	

Parameter	Nilai	Keterangan
		Overlap antar sub-chunk jika section terlalu panjang

Alur chunking per file:

1. Baca file `.txt`, skip blok TOC (deteksi dengan pola `\t<angka>` di akhir baris).
2. Parse heading section dengan `SECTION_RE` → dapatkan `(section_num, section_title, body_lines)`.
3. Jika body section > `MAX_CHARS`, sub-split dengan overlap 200 char.
4. Tiap chunk diberi metadata wajib.

Metadata tiap chunk:

```
{
  "spec_id":      "TS 24.008",
  "version":      "g50",          # kode rilis 3GPP dari nama file
  "section":      "4.7.3",
  "section_title": "GPRS attach procedure",
  "source_file":  "24008-g50.txt",
  "chunk_index":  0               # indeks sub-chunk jika section di-split
}
```

Chroma ID format:

```
TS_24.008|g50|4.7.3|0
```

6. Multi-Topik & Skalabilitas

6.1 Konsep Topik

NORA adalah **engine RAG generik** yang melayani banyak knowledge base. Tiap knowledge base disebut **Topik**. Tiap Topik memiliki collection ChromaDB sendiri namun berbagi seluruh pipeline RAG yang sama.

Model data Topik (PostgreSQL, tabel `topics`):

```
class Topic(Base):
    id          # UUID hex
    slug        # mis. "3gpp-ts24008" – unik, dipakai sebagai key
    name        # mis. "3GPP TS 24.008"
    description  # ringkasan domain
    collection_name # nama collection di ChromaDB: "ts24008"
    created_at
```

6.2 Roadmap Topik

Topik	Status	Domain
3GPP TS 24.008	● Live (terbukti E2E)	NAS Layer 3 — MM / GMM / CC / SM
3GPP TS 23.501	◦ Planned	5G System Architecture
3GPP TS 33.501	◦ Planned	5G Security

Topik	Status	Domain
ITU-T (seri Q/X)	◦ Planned	Signalling & data networks
IEEE 802.x	◦ Planned	LAN / Wireless
GSMA / O-RAN	◦ Planned	Operator & Open RAN

6.3 Alur Penambahan Topik Baru

```

[Admin] registrasi metadata Topik (slug, name, description, collection_name)
↓
Sistem seed ke tabel topics (PostgreSQL)
↓
[Admin] jalankan ingest: point ke folder .txt knowledge base Topik
↓
Chunker section-aware memproses file → embed → upsert ke ChromaDB collection baru
↓
Topik status → "live" – siap dipakai user, tanpa ubah kode engine

```

6.4 Skalabilitas Arsitektur

Aspek	Pendekatan
Multi-tenant	Stateless per-request; state (memory, history) di Postgres — aman di-load-balance
Isolasi data	Tiap Topik = collection ChromaDB sendiri; retrieval tidak pernah lintas-collection
Scale horizontal	Backend FastAPI dapat di-replikasi; state tidak di-memory proses
Upgrade vector store	Demo: ChromaDB embedded → Produksi: swap ke Qdrant tanpa ubah arsitektur
RAM guard	Container backend dibatasi <code>mem_limit: 3g</code> (Docker Compose) — cegah OOM RPi5

7. Keamanan & Auth

7.1 Autentikasi

Mekanisme	Detail
Password hashing	<code>bcrypt</code> via <code>passlib</code> — hash one-way, tidak pernah simpan plaintext
Token	JWT (JSON Web Token), algoritma HS256, masa berlaku 7 hari (<code>COOKIE_MAX_AGE = 7 * 24 * 3600</code>)
Cookie	<code>httpOnly=True</code> (tidak bisa diakses JavaScript), <code>SameSite=lax</code> , <code>Secure=True</code> di produksi
Secret key	<code>SECRET_KEY</code> via environment variable — aplikasi fail-fast jika tidak di-set di produksi

Flow autentikasi:

```

POST /api/auth/register → bcrypt hash password → simpan ke users (Postgres)
POST /api/auth/login    → verifikasi bcrypt → buat JWT → set httpOnly cookie
GET  /api/*             → middleware baca cookie → decode JWT → inject user ke route
POST /api/auth/logout   → delete cookie

```

7.2 Input Validation

Parameter	Batasan
<code>message</code> (query)	Minimal 1 karakter, maksimal 4000 karakter
<code>top_k</code>	Minimal 1, maksimal 20
<code>topic_id</code>	Wajib ada, divalidasi ke tabel <code>topics</code>
<code>session_id</code>	Jika disuplai, harus milik user yang sama (cek <code>user_id</code>)

7.3 Security Headers

Konfigurasi produksi mengaktifkan security headers standar:

Header	Nilai	Tujuan
<code>Strict-Transport-Security</code>	<code>max-age=31536000; includeSubDomains</code>	HSTS — paksa HTTPS
<code>Content-Security-Policy</code>	Dibatasi ke origin yang diizinkan	Cegah XSS
<code>X-Frame-Options</code>	<code>DENY</code>	Cegah clickjacking
<code>X-Content-Type-Options</code>	<code>nosniff</code>	Cegah MIME sniffing

7.4 Akses Remote

Akses publik ke `nora.arahkarya.com` melalui **Cloudflare Tunnel** — tidak ada port yang terbuka langsung ke internet di host RPi5. Semua traffic dienkripsi dan difilter oleh Cloudflare sebelum sampai ke server.

7.5 CORS

Origins yang diizinkan dikonfigurasi via `NORA_CORS_ORIGINS` :

```
NORA_CORS_ORIGINS=http://localhost:3030,https://nora.arahkarya.com
```

8. Deployment

8.1 Stack Docker Compose

NORA dijalankan dengan tiga service Docker Compose:

```

# docker-compose.yml
services:
  postgres: # state & auth
  backend:  # FastAPI + ChromaDB embedded
  frontend: # Next.js 14

```

Port mapping:

Service	Port Host	Port Container	Keterangan
frontend	3030	3000	Next.js dashboard
backend	8010	8000	FastAPI API
postgres	5440	5432	PostgreSQL 16
ChromaDB	—	—	Embedded di backend, tidak expose port

Jaringan internal:

```
nora-net (bridge) – subnet 10.0.14.0/24
```

Seluruh service berkomunikasi lewat network `nora-net` . Backend mengakses 9router dan Ollama di host melalui `host.docker.internal` .

8.2 Quickstart

```
# Clone & masuk ke direktori
cd /home/yay/apps/nora

# Salin dan edit konfigurasi
cp .env.example .env
# Edit: SECRET_KEY, POSTGRES_PASSWORD, NORA_ROUTER_URL, dll.

# Build dan jalankan semua service
docker compose up -d --build

# Cek status
docker compose ps
docker compose logs -f backend
```

8.3 Environment Variables Utama

Variabel	Default	Keterangan
SECRET_KEY	(wajib di-set)	Secret JWT — fail-fast jika dev default di produksi
DATABASE_URL	postgresql+psycpg://nora:...@postgres:5432/nora	Koneksi PostgreSQL
NORA_ENGINE	9router	Engine LLM: 9router ollama
NORA_GEN_MODEL	cc/claude-opus-4-8	Model generator
NORA_VERIFY_MODEL	cc/claude-sonnet-4-6	Model verifier
NORA_EMBED_BACKEND	9router	Backend embedding: 9router ollama local
NORA_EMBED_MODEL	gemini/gemini-embedding-001	Model embedding (dim 3072 untuk 9router)
NORA_ROUTER_URL	http://host.docker.internal:20128/v1	URL 9router (OpenAI-compatible)

Variabel	Default	Keterangan
NORA_CHROMA_PATH	/app/data/chroma	Path ChromaDB di dalam container
NORA_COLLECTION	ts24008	Collection ChromaDB default
NORA_CORS_ORIGINS	http://localhost:3030,https://nora.arahkarya.com	Origins CORS yang diizinkan
NORA_COOKIE_SECURE	true	Set <code>false</code> untuk development HTTP
NORA_ENABLE_DOCS	false	Set <code>true</code> untuk aktifkan Swagger/ReDoc

8.4 Ingest Knowledge Base

```
# Masuk ke environment backend
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Test ingest 1 versi (verifikasi pipeline)
python -m nora.ingest.run --limit 1

# Ingest semua 257 versi TS 24.008
python -m nora.ingest.run

# Cek statistik indeks
python -c "from nora.rag.store import stats; print(stats())"
```

8.5 Cloudflare Tunnel

Akses publik NORA menggunakan Cloudflare Tunnel — tidak ada port yang dibuka ke internet:

```
Internet → Cloudflare Edge → Tunnel → host RPi5 → nora-frontend:3030
                                     ↗ /api/* → nora-backend:8010
```

- Domain: `nora.arahkarya.com`
- Frontend (port `3030`) melayani semua request; path `/api/*` di-proxy ke backend internal via Next.js rewrites.
- Cookie `Secure=true` karena semua traffic masuk via HTTPS Cloudflare.

8.6 Topologi Produksi vs Demo

Aspek	Demo (RPi5 saat ini)	Produksi (target)
Vector store	ChromaDB embedded	Qdrant (swap via config, tanpa rewrite)
LLM	9router cloud (Opus/Sonnet)	9router cloud atau vLLM self-hosted
Backend replika	1 instance	N replika + load balancer
Database	PostgreSQL container	PostgreSQL managed / dedicated
Hardware	Raspberry Pi 5 (8GB)	VPS / cloud server

Lampiran A: Struktur Repositori

```
nora/
├── docs/
│   ├── BRD-NORA.md          # Business Requirements Document
│   ├── PRD-NORA.md          # Product Requirements Document
│   ├── NORA-Overview.md     # Dokumen ini
│   └── screenshots/         # Tampilan UI (login, workspace, jawaban)
├── backend/
│   └── nora/
│       ├── main.py           # FastAPI app entrypoint, startup seed
│       ├── api/
│       │   ├── routes_auth.py    # Register, login, logout, me
│       │   ├── routes_query.py   # POST /api/query – main RAG endpoint
│       │   ├── routes_topics.py  # GET /api/topics
│       │   └── routes_history.py # GET /api/history
│       ├── pipeline/
│       │   ├── orchestrator.py   # NORA Agent Layer: alur utama RAG
│       │   ├── reason.py         # Generator + Verifier + fallback chain
│       │   └── validate.py       # Confidence scoring + flag
│       ├── rag/
│       │   └── store.py          # ChromaDB: add_chunks, retrieve, stats
│       ├── engine/
│       │   └── config.py         # Env config, LLM client, embed_texts()
│       ├── ingest/
│       │   └── chunker.py        # Section-aware chunker 3GPP
│       ├── auth/
│       │   ├── security.py       # bcrypt, JWT create/verify
│       │   └── deps.py           # FastAPI dependency: get_current_user
│       └── db/
│           ├── database.py       # SQLAlchemy engine, Base, get_db
│           └── models.py         # ORM: User, Topic, ChatSession, Message
├── frontend/                  # Next.js 14 (auth context, sources panel)
└── docker-compose.yml        # Postgres + backend + frontend
```

Lampiran B: Contoh Respons API

POST /api/query

Request:

```
{
  "topic_id": "<uuid-topik-ts24008>",
  "message": "Jelaskan prosedur GPRS Attach di TS 24.008",
  "top_k": 5
}
```

Response:

```
{
  "answer": "Prosedur GPRS Attach dimulai saat MS mengirim ATTACH REQUEST ke SGSN...",
  "confidence": 0.87,
  "flag": null,
  "verifier_verdict": "VALID",
  "verifier_reason": "Semua klaim didukung penuh oleh konteks §4.7.3.1",
}
```



```

"regeneration_count": 0,
"sources": [
  {
    "spec": "TS 24.008",
    "version": "g50",
    "section": "4.7.3",
    "title": "GPRS attach procedure",
    "similarity": 0.9241,
    "chunk_text": "[TS 24.008 vg50 §4.7.3] GPRS attach procedure\nThe GPRS attach..."
  }
],
"query_id": "q_a1b2c3d4",
"session_id": "<uuid-session>"
}

```

Contoh Perilaku Anti-Halusinasi

Query: "Berapa harga iPhone terbaru?"

Response:

```

{
  "answer": "Informasi tidak ditemukan dalam spec yang tersedia.",
  "confidence": 0.0,
  "flag": "NO CONTEXT",
  "verifier_verdict": "INVALID",
  "sources": [],
  "regeneration_count": 0
}

```

© 2026 PT Arah Karya Sinergi × NOZ

BAGIAN II

Tutorial Operasional

Setup dari nol, ingest knowledge base, query, ganti embedding backend, operasional harian & troubleshooting.

NORA — Tutorial Operasional Lengkap

Network Oracle for Reliable Answers

Disiapkan oleh	Tim ArahKarya
Kolaborasi	PT Arah Karya Sinergi × NOZ
Tanggal	Juni 2026
Versi dokumen	1.0
Platform	RAG Engine Telekomunikasi
Topik aktif	3GPP TS 24.008 (257 versi, R98-R18)

Daftar Isi

1. Prasyarat
2. Setup dari Nol
3. Ingest Knowledge Base
4. Query NORA
5. Ganti Embedding Backend
6. Re-Ingest Portable ke Server Baru
7. Operasional Harian
8. Troubleshooting

1. Prasyarat

1.1 Perangkat Keras

Komponen	Minimum	Rekomendasi (produksi)
CPU	ARM64 / x86-64, 4 core	Raspberry Pi 5 8GB / VPS 4 vCPU
RAM	4 GB	8 GB (RPi5 produksi saat ini)
Disk	20 GB	40 GB+ (ChromaDB bisa capai 7-8 GB)
OS	Linux 64-bit	Debian 12 / Ubuntu 22.04 / Raspberry Pi OS 64-bit

Catatan RAM — RPi5 8 GB: ChromaDB [mmap](#) bisa makan hingga 7,6 GB saat index penuh. Sistem produksi NORA memakai mitigasi berikut — **pastikan semua sudah aktif sebelum ingest berat:**

- [earlyoom](#) daemon aktif (mencegah freeze/reboot karena OOM)
- Swap file 6 GB ([/swapfile](#))
- [vm.swappiness=30](#) di [/etc/sysctl.conf](#)
- Docker [mem_limit: 3g](#) pada container [nora-backend](#) (OOM-kill container, bukan host)

Jangan jalankan ingest berat bersamaan dengan app produksi di RAM mepet.

1.2 Perangkat Lunak

Perangkat Lunak	Versi	Catatan
Docker Engine	≥ 24	<code>docker compose</code> (plugin V2)
Docker Compose plugin	≥ 2.20	bawaan Docker Engine modern
Python	3.11	untuk CLI ingest & query
Git	≥ 2.40	clone repo
<code>curl</code>	—	health check
Ollama	≥ 0.3	opsional — hanya jika <code>NORA_EMBED_BACKEND=ollama</code>

Cek versi:

```
docker --version
docker compose version
python3 --version
git --version
```

1.3 Akses Jaringan

Kebutuhan	Diperlukan Oleh
Akses internet ke cloud LLM via 9router	Default <code>NORA_EMBED_BACKEND=9router</code> + LLM generator/verifier
9router berjalan di port <code>20128</code> (host atau VPN)	<code>NORA_ROUTER_URL=http://host.docker.internal:20128/v1</code>
Tidak perlu internet	Mode <code>NORA_EMBED_BACKEND=local</code> (fastembed offline)

2. Setup dari Nol

2.1 Clone Repositori

```
git clone https://github.com/ArahKarya/nora.git ~/apps/nora
cd ~/apps/nora
```

Repositori sudah menyertakan **257 file .txt knowledge base 3GPP TS 24.008** di `data/3gpp/24008/txt/` — tidak perlu unduh terpisah.

2.2 Konfigurasi Environment

Salin template environment dan sesuaikan:

```
cd ~/apps/nora
cp .env.example .env
nano .env          # atau editor pilihan Anda
```

Variabel env wajib diisi:

```
# ===== NORA - environment =====

# --- Postgres ---
POSTGRES_USER=nora
POSTGRES_PASSWORD=GANTI_PASSWORD_KUAT_INI
POSTGRES_DB=nora

# --- Backend (FastAPI) ---
DATABASE_URL=postgresql+psycopg://nora:GANTI_PASSWORD_KUAT_INI@postgres:5432/nora
SECRET_KEY=GANTI_DENGAN_openssl_rand_hex_32 # wajib ganti di produksi!
NORA_CORS_ORIGINS=http://localhost:3030,https://nora.arahkarya.com

# --- LLM via 9router ---
NORA_ROUTER_URL=http://host.docker.internal:20128/v1
NORA_ROUTER_KEY=sk-local
NORA_GEN_MODEL=cc/claude-opus-4-8
NORA_VERIFY_MODEL=cc/claude-sonnet-4-6

# --- Embedding backend (pilih salah satu, lihat Bab 5) ---
NORA_EMBED_BACKEND=9router
NORA_EMBED_MODEL=gemini/gemini-embedding-001

# --- ChromaDB ---
NORA_CHROMA_PATH=/app/data/chroma
NORA_COLLECTION=ts24008
```

Generate `SECRET_KEY` yang aman:

```
openssl rand -hex 32
```

Salin hasilnya ke nilai `SECRET_KEY` di `.env`.

2.3 Build dan Jalankan Stack Docker

```
cd ~/apps/nora
docker compose up -d --build
```

Proses build pertama memerlukan beberapa menit (download image + build Next.js). Build ulang hanya diperlukan jika ada perubahan kode.

Port yang dipakai:

Service	Container	Port Host	Keterangan
nora-frontend	frontend:3000	3030	UI Next.js
nora-backend	backend:8000	8010	FastAPI + RAG
nora-postgres	postgres:5432	5440	PostgreSQL state

2.4 Cek Health Stack

```
# Status semua container
docker compose ps

# Cek backend API
curl -s http://localhost:8010/healthz

# Cek frontend
curl -s -o /dev/null -w "%{http_code}" http://localhost:3030
```

Output `docker compose ps` yang normal:

NAME	IMAGE	STATUS	PORTS
nora-backend	nora-backend	Up (healthy)	0.0.0.0:8010->8000/tcp
nora-frontend	nora-frontend	Up	0.0.0.0:3030->3000/tcp
nora-postgres	postgres:16-alpine	Up (healthy)	0.0.0.0:5440->5432/tcp

2.5 Setup Python Virtual Environment (untuk CLI)

CLI ingest dan query berjalan **di luar Docker**, langsung di host:

```
cd ~/apps/nora/backend
python3 -m venv .env
source .env/bin/activate
pip install -r requirements.txt
```

3. Ingest Knowledge Base

Ingest = proses membaca file `.txt` 3GPP → chunk per-section → embed → simpan ke ChromaDB.

Knowledge base yang tersedia:

```
~/apps/nora/data/3gpp/24008/txt/
└─ 257 file .txt (3GPP TS 24.008, versi R98 1999 – R18 2026)
```

3.1 Test Ingest (1 Versi)

Selalu mulai dengan test 1 versi untuk verifikasi koneksi embedding berjalan:

```
cd ~/apps/nora/backend
source .env/bin/activate

python -m nora.ingest.run --limit 1
```

Output normal:

```
[ingest] 1 file dari /home/yay/apps/nora/data/3gpp/24008/txt
[1/1] 24008-100.txt: 1051 chunks (total 1051)
[ingest] DONE 1051 chunks baru, 0 versi di-skip, dalam 42s. collection=ts24008 total=1051
```

Jika berhasil, lanjut ke ingest penuh.

3.2 Ingest Penuh (257 Versi)

⚠ **Peringatan RAM:** Ingest penuh 257 versi berat di RAM. Di RPi5 8 GB, **jangan** jalankan bersamaan dengan traffic produksi. Gunakan `systemd-run` agar proses tetap hidup saat terminal ditutup:

```
cd ~/apps/nora/backend
source .venv/bin/activate

# Ingest semua (bisa berjam-jam di RPi5)
python -m nora.ingest.run
```

Atau via `systemd-run` agar tidak mati saat terminal tutup (disarankan di RPi5):

```
systemd-run --user --no-block \
  bash -c "cd ~/apps/nora/backend && source .venv/bin/activate && python -m nora.ingest.run"
```

3.3 Ingest File Spesifik

Untuk ingest versi tertentu saja:

```
cd ~/apps/nora/backend
source .venv/bin/activate

python -m nora.ingest.run --files 24008-g50.txt 24008-j50.txt
```

3.4 Resume Ingest (Skip Versi Sudah Ada)

Jika ingest terganggu di tengah jalan, lanjutkan tanpa mengulang dari awal:

```
python -m nora.ingest.run --skip-existing
```

Flag `--skip-existing` membaca daftar versi yang sudah terindeks di ChromaDB dan melewatinya.

3.5 Parameter Lengkap CLI Ingest

Parameter	Default	Keterangan
(<i>tanpa parameter</i>)	—	Ingest semua <code>.txt</code> di <code>KB_TXT_DIR</code>
<code>--limit N</code>	0 (semua)	Batasi N file pertama (untuk test)
<code>--files F1 F2 ...</code>	—	Nama file spesifik di folder KB
<code>--dir PATH</code>	<code>NORA_KB_DIR</code>	Override direktori sumber
<code>--skip-existing</code>	<code>false</code>	Lewati versi yang sudah terindeks

4. Query NORA

4.1 Via UI Web (Rekomendasi)

Akses Antarmuka

Lingkungan	URL
Produksi (via Cloudflare tunnel)	https://nora.arahkarya.com
Lokal	http://localhost:3030

Langkah Query via UI

Langkah 1 — Login

Buka URL di atas. Masukkan email dan password akun Anda. Contoh akun: yayang@arahkarya.com

Langkah 2 — Pilih Topik

Setelah login, panel kiri menampilkan daftar **Topik** yang tersedia. Klik "**3GPP TS 24.008**" untuk memulai sesi pada knowledge base NAS L3.

Langkah 3 — Ketik Pertanyaan

Di panel tengah (area chat), ketik pertanyaan teknis pada kotak input di bagian bawah. Contoh pertanyaan:

Apa itu prosedur GPRS Attach?

Jelaskan mekanisme paging di 3GPP TS 24.008 §6.5

Apa bedanya GMM Attach dan MM Location Update?

Tekan **Enter** atau klik tombol kirim.

Langkah 4 — Baca Jawaban

NORA memproses query melalui pipeline: [embed query](#) → [retrieve top-K](#) → [Generator \(Opus\)](#) → [Verifier \(Sonnet\)](#) → [validasi](#)

Jawaban ditampilkan dengan:

- **Teks jawaban** tergrounded pada spec resmi
- **Confidence score** — [TINGGI \(≥0.7\)](#) ● / [SEDANG](#) ● / [RENDAH](#) □
- **Verifier verdict** — [VALID](#) / [INVALID](#) (jika INVALID, pipeline regen otomatis)
- **Panel Sumber** (kanan) — daftar section spec yang dirujuk, lengkap dengan §section dan similarity score

Langkah 5 — Telusuri Sumber

Panel **Sumber** di kanan menampilkan referensi seperti:

- ▶ TS 24.008 v16.7.0 §4.7.3.1 – GPRS Attach procedure (sim 0.94)
- ▶ TS 24.008 v15.6.0 §4.7.3.2 – Attach reject (sim 0.87)

Klik sumber untuk melihat kutipan teks asli dari spec.

Pertanyaan di luar cakupan spec akan dijawab: "*Informasi tidak ditemukan dalam spec.*" — NORA tidak mengarang jawaban.

4.2 Via CLI (Command Line)

Query Dasar

```
cd ~/apps/nora/backend
source .venv/bin/activate

python -m nora.ask "Apa itu prosedur GPRS Attach?"
```

Output contoh:

```
=====
Q: Apa itu prosedur GPRS Attach?

Prosedur GPRS Attach adalah mekanisme yang digunakan oleh MS (Mobile Station)
untuk mendaftarkan diri ke jaringan GPRS. Prosedur ini dijelaskan dalam
§4.7.3.1 TS 24.008 dan mencakup...

Confidence: ● 0.93 | Verifier: VALID | regen=0

Sumber:
▶ TS 24.008 v16.7.0 §4.7.3.1 – GPRS attach procedure (sim 0.94)
▶ TS 24.008 v15.6.0 §4.7.3 – General (sim 0.89)
▶ TS 24.008 v14.5.0 §4.7.3.1 – GPRS attach procedure (sim 0.87)
▶ TS 24.008 v13.3.0 §4.7.4 – GPRS detach procedure (sim 0.81)
▶ TS 24.008 v12.2.0 §4.7.3.2 – GPRS attach procedure (sim 0.79)
=====
```

Parameter CLI Query

Parameter	Default	Keterangan
<code>query</code>	<i>(wajib)</i>	Pertanyaan (string, dalam tanda kutip)
<code>--top-k N</code>	5	Jumlah dokumen yang diambil dari ChromaDB
<code>--version VER</code>	None (semua)	Filter versi spesifik, mis. 24008-g50
<code>--json</code>	false	Output JSON terstruktur (untuk integrasi)

Contoh Penggunaan CLI

```
# Query dengan filter versi tertentu
python -m nora.ask --version 24008-g50 "Apa itu Attach Request?"
```

```
# Ambil 10 sumber teratas
python -m nora.ask --top-k 10 "Jelaskan GMM state machine"

# Output JSON (untuk parsing/integrasi)
python -m nora.ask --json "Apa itu paging procedure?" | python -m json.tool
```

5. Ganti Embedding Backend

5.1 Perbandingan Backend Embedding

Backend	Nilai NORA_EMBED_BACKEND	Dimensi Vektor	Koneksi	Kapan Dipakai
9router + Gemini	9router	3072	Cloud (internet + 9router)	Default produksi. Kualitas terbaik.
Ollama lokal	ollama	768 (nomic) / 1024 (mxbai)	Lokal (Ollama daemon)	Server tanpa internet, ada GPU/ CPU kuat.
fastembed lokal	local	384	Offline CPU	Paling ringan. Dev/ test tanpa koneksi apapun.

⚠ **ATURAN KRITIS:** Index ChromaDB dan proses query **wajib menggunakan backend + model embedding yang sama**. Dimensi vektor berbeda → ChromaDB error dimensi tidak cocok. **Ganti backend = wajib re-ingest ulang dari awal.**

5.2 Cara Ganti Backend

Langkah-Langkah Ganti Embedding Backend

Langkah 1 — Edit .env

```
nano ~/apps/nora/.env
```

Ubah baris berikut sesuai pilihan (lihat tabel di atas):

```
# Pilihan 1: 9router Gemini (default produksi)
NORA_EMBED_BACKEND=9router
NORA_EMBED_MODEL=gemini/gemini-embedding-001

# Pilihan 2: Ollama lokal
NORA_EMBED_BACKEND=ollama
NORA_EMBED_OLLAMA_MODEL=nomic-embed-text
NORA_OLLAMA_URL=http://host.docker.internal:11434/v1

# Pilihan 3: fastembed (offline CPU)
NORA_EMBED_BACKEND=local
NORA_EMBED_LOCAL_MODEL=BAAI/bge-small-en-v1.5
```

Langkah 2 — (Jika Ollama) Siapkan Model

```
# Pastikan Ollama daemon berjalan
ollama serve &

# Pull model embedding
ollama pull nomic-embed-text      # dim 768, rekomendasi
# atau
ollama pull mxbai-embed-large     # dim 1024, lebih akurat
```

Langkah 3 — Hapus Collection ChromaDB Lama

Karena dimensi berbeda, collection lama tidak kompatibel:

```
# Hapus data ChromaDB lama
rm -rf ~/apps/nora/backend/data/chroma/*

# Atau hapus collection spesifik via Python
cd ~/apps/nora/backend
source .venv/bin/activate
python - <<'EOF'
import chromadb
client = chromadb.PersistentClient(path="./data/chroma")
client.delete_collection("ts24008")
print("Collection ts24008 dihapus.")
EOF
```

Langkah 4 — Restart Container Backend

```
cd ~/apps/nora
docker compose restart backend
```

Langkah 5 — Re-Ingest Knowledge Base

```
cd ~/apps/nora/backend
source .venv/bin/activate

# Ekspor env baru (samakan dengan .env)
export NORA_EMBED_BACKEND=ollama      # sesuaikan
export NORA_EMBED_OLLAMA_MODEL=nomic-embed-text

# Test 1 versi dulu
python -m nora.ingest.run --limit 1

# Jika OK, ingest penuh
python -m nora.ingest.run
```

5.3 Env Lengkap per Backend

Env Var	9router	ollama	local
NORA_EMBED_BACKEND	9router	ollama	local
NORA_EMBED_MODEL	gemini/gemini-embedding-001	(tidak dipakai)	(tidak dipakai)
NORA_EMBED_OLLAMA_MODEL	(tidak dipakai)	nomic-embed-text	(tidak dipakai)
NORA_OLLAMA_URL	(tidak dipakai)		(tidak dipakai)

Env Var	9router	ollama	local
		http:// host.docker.internal:11434/ v1	
NORA_EMBED_LOCAL_MODEL	(tidak dipakai)	(tidak dipakai)	BAAI/bge-small-en-v1.5
NORA_ROUTER_URL	http:// host.docker.internal:20128/ v1	(tidak dipakai untuk embed)	(tidak dipakai)
NORA_ROUTER_KEY	sk-local	(tidak dipakai untuk embed)	(tidak dipakai)

6. Re-Ingest Portable ke Server Baru

NORA dirancang **portable** — knowledge base (257 file `.txt`) sudah ada di repo, bukan di ChromaDB yang besar. Pindah server = clone repo + re-ingest, **tanpa perlu copy ChromaDB 7,6 GB**.

6.1 Menggunakan Skrip `reingest.sh`

```
# Clone repo di server baru
git clone https://github.com/ArahKarya/nora.git ~/apps/nora
cd ~/apps/nora

# Jalankan skrip re-ingest (default: Ollama offline)
bash scripts/reingest.sh
```

Skrip melakukan:

1. Validasi 257 file `.txt` ada di repo
2. Buat Python venv + install dependencies
3. (Jika Ollama) Cek Ollama daemon aktif + pull model jika belum ada
4. Test ingest 1 versi — **minta konfirmasi** sebelum lanjut
5. Ingest penuh semua versi dengan `--skip-existing`

6.2 Pilih Backend Saat Re-Ingest

Override backend via env sebelum jalankan skrip:

```
# Gunakan 9router Gemini (cloud)
NORA_EMBED_BACKEND=9router \
NORA_ROUTER_URL=http://localhost:20128/v1 \
NORA_ROUTER_KEY=sk-local \
bash scripts/reingest.sh

# Gunakan Ollama offline (default skrip)
NORA_EMBED_BACKEND=ollama \
NORA_EMBED_OLLAMA_MODEL=nomic-embed-text \
bash scripts/reingest.sh

# Gunakan fastembed lokal (paling ringan)
NORA_EMBED_BACKEND=local \
bash scripts/reingest.sh
```

6.3 Checklist Migrasi Server Baru

Ikuti urutan ini saat pindah server:

1. Install Docker Engine + Docker Compose plugin
2. Clone repo: `git clone ... ~/apps/nora`
3. Salin dan edit `.env`: `cp .env.example .env && nano .env`
4. (Opsional) Jika Ollama: `ollama serve` + `ollama pull nomic-embed-text`
5. Jalankan stack: `cd ~/apps/nora && docker compose up -d --build`
6. Setup Python venv: `cd backend && python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt`
7. Re-ingest: `bash scripts/reingest.sh` atau `python -m nora.ingest.run`
8. Cek health: `docker compose ps` + `curl http://localhost:8010/healthz`
9. Cek query: `python -m nora.ask "Apa itu GPRS Attach?"`

7. Operasional Harian

7.1 Start, Stop, Restart Stack

```
cd ~/apps/nora

# Jalankan semua service (tanpa build ulang)
docker compose start

# Hentikan semua service (data tetap aman)
docker compose stop

# Restart semua service
docker compose restart

# Restart service tertentu saja
docker compose restart backend
docker compose restart frontend

# Lihat status semua container
docker compose ps

# Matikan dan hapus container (data volume tetap aman)
docker compose down

# Matikan dan hapus container + volume (DATA TERHAPUS – hati-hati!)
# docker compose down -v
```

7.2 Melihat Log

```
cd ~/apps/nora

# Log semua service (live, ikuti terus)
docker compose logs -f

# Log service tertentu
docker compose logs -f backend
docker compose logs -f frontend
```

```
docker compose logs -f postgres

# Log 100 baris terakhir tanpa follow
docker compose logs --tail=100 backend
```

7.3 Masuk ke Shell Container

```
# Shell di container backend
docker exec -it nora-backend bash

# Shell di container postgres
docker exec -it nora-postgres psql -U nora -d nora
```

7.4 Cek Statistik ChromaDB

```
cd ~/apps/nora/backend
source .venv/bin/activate

python - <<'EOF'
import chromadb
client = chromadb.PersistentClient(path="./data/chroma")
col = client.get_collection("ts24008")
print(f"Collection: ts24008")
print(f"Total chunks: {col.count()}")
EOF
```

7.5 Backup

Backup PostgreSQL (Wajib Rutin)

PostgreSQL menyimpan: user, sesi chat, history query, registrasi topik.

```
# Dump database
docker exec nora-postgres pg_dump -U nora nora > ~/backup/nora-pg-$(date +%Y%m%d).sql

# Restore dari dump
docker exec -i nora-postgres psql -U nora nora < ~/backup/nora-pg-20260615.sql
```

Backup ChromaDB (Opsional — bisa re-ingest)

ChromaDB bisa dibangun ulang dari file `.txt` kapan saja (via re-ingest). Backup hanya jika ingin hemat waktu ingest ulang:

```
# ChromaDB ada di mount volume host
tar -czf ~/backup/nora-chroma-$(date +%Y%m%d).tar.gz \
~/apps/nora/backend/data/chroma/
```

Backup File `.env`

```
# Simpan .env di tempat aman (jangan di repo!)
cp ~/apps/nora/.env ~/backup/nora-env-$(date +%Y%m%d).bak
```

7.6 Update Aplikasi

```
cd ~/apps/nora

# Ambil update kode terbaru
git pull

# Build ulang image dan restart
docker compose up -d --build

# Cek tidak ada container yang gagal
docker compose ps
```

8. Troubleshooting

8.1 RAM Mepet / OOM / Host Reboot

Gejala: Host tiba-tiba reboot, container mati mendadak, log kernel `oom-kill`.

Penyebab: ChromaDB `mmap` membaca semua chunk ke RAM (bisa 7,6 GB untuk 257 versi).

Solusi:

```
# 1. Cek penggunaan RAM saat ini
free -h
docker stats --no-stream

# 2. Pastikan earlyoom aktif
systemctl status earlyoom

# Jika belum install:
sudo apt install earlyoom
sudo systemctl enable --now earlyoom

# 3. Cek swap
swapon --show

# Jika belum ada swap 6 GB:
sudo fallocate -l 6G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
# Tambah ke /etc/fstab agar persisten:
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

# 4. Set swappiness rendah
echo 'vm.swappiness=30' | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# 5. Jangan ingest berat saat produksi aktif
# Gunakan systemd-run untuk ingest background:
systemd-run --user --no-block \
    bash -c "cd ~/apps/nora/backend && source .venv/bin/activate && python -m nora.ingest.run"
```

8.2 Error Dimensi Vektor Tidak Cocok

Gejala:

```
chromadb.errors.InvalidDimensionException: Embedding dimension 768 does not match  
collection dimensionality 3072
```

Penyebab: Backend embedding diganti tanpa re-ingest. Koleksi ChromaDB menyimpan vektor dengan dimensi lama, query memakai dimensi baru.

Solusi:

```
# 1. Hapus collection lama  
cd ~/apps/nora/backend  
source .venv/bin/activate  
  
python - <<'EOF'  
import chromadb  
client = chromadb.PersistentClient(path="./data/chroma")  
client.delete_collection("ts24008")  
print("Hapus OK.")  
EOF  
  
# 2. Pastikan .env sudah di-set ke backend baru  
nano ~/apps/nora/.env  
  
# 3. Restart backend  
cd ~/apps/nora  
docker compose restart backend  
  
# 4. Re-ingest dengan backend baru  
cd backend  
python -m nora.ingest.run --limit 1 # test dulu  
python -m nora.ingest.run          # ingest penuh
```

8.3 Container Tidak Jalan / Exit

Gejala: `docker compose ps` menampilkan status `Exit` atau `Restarting`.

Langkah diagnosis:

```
cd ~/apps/nora  
  
# Lihat log error container yang bermasalah  
docker compose logs --tail=50 backend  
docker compose logs --tail=50 frontend  
docker compose logs --tail=50 postgres  
  
# Cek apakah postgres healthy (backend depends_on postgres)  
docker compose ps postgres  
  
# Coba restart  
docker compose restart backend
```

Masalah umum:

Error Log	Penyebab	Solusi
password authentication failed	POSTGRES_PASSWORD di .env tidak sinkron	Samakan password di semua variabel .env
could not connect to server	Backend start sebelum postgres siap	docker compose restart backend
Secret key not set	SECRET_KEY masih nilai default atau kosong	Generate dengan openssl rand -hex 32
Address already in use :3030	Port 3030 dipakai proses lain	Ganti port host di docker-compose.yml atau matikan proses lain
NORA_ROUTER_URL connection refused	9router tidak berjalan di port 20128	Jalankan 9router di host, atau ganti ke backend local

8.4 Cloudflare Tunnel Down

Gejala: <https://nora.arahkarya.com> tidak bisa diakses, UI lokal <http://localhost:3030> masih hidup.

```
# Cek status cloudflared
systemctl status cloudflared

# Lihat log tunnel
journalctl -u cloudflared -n 50 --no-pager

# Restart tunnel
sudo systemctl restart cloudflared

# Verifikasi tunnel terdaftar di Cloudflare Zero Trust dashboard
# https://one.dash.cloudflare.com → Networks → Tunnels
```

Jika tunnel OK tapi nora.arahkarya.com masih tidak jalan, cek:

```
# Pastikan frontend container berjalan
docker compose ps frontend

# Pastikan tunnel pointing ke port 3030
# (config di Cloudflare dashboard: http://localhost:3030)
```

8.5 Terminal RPi5: "open terminal failed: not a terminal"

Gejala: Background process gagal saat jalankan via terminal session tertentu di RPi5.

Solusi: Gunakan `systemd-run` untuk proses panjang:

```
# Cara aman jalankan ingest panjang di RPi5
systemd-run --user --no-block \
  bash -c "cd ~/apps/nora/backend && source .venv/bin/activate && python -m nora.ingest.run"

# Pantau progress
journalctl --user -f
```

8.6 Query CLI Error: "Collection not found"

Gejala:

```
chromadb.errors.InvalidCollectionException: Collection ts24008 does not exist.
```

Penyebab: ChromaDB belum pernah di-ingest, atau collection terhapus.

Solusi: Jalankan ingest terlebih dahulu (lihat [Bab 3](#)).

```
cd ~/apps/nora/backend
source .venv/bin/activate
python -m nora.ingest.run --limit 1 # minimal test 1 versi
```

8.7 Ingest Gagal di Tengah Jalan

Gejala: Error network/timeout saat ingest, proses berhenti sebelum selesai.

Solusi: Gunakan `--skip-existing` untuk melanjutkan tanpa mengulang:

```
cd ~/apps/nora/backend
source .venv/bin/activate
python -m nora.ingest.run --skip-existing
```

Referensi Cepat

Perintah Paling Sering Dipakai

```
# Start stack
cd ~/apps/nora && docker compose start

# Stop stack
cd ~/apps/nora && docker compose stop

# Status
cd ~/apps/nora && docker compose ps

# Log backend live
cd ~/apps/nora && docker compose logs -f backend

# Query CLI cepat
cd ~/apps/nora/backend && source .venv/bin/activate && \
python -m nora.ask "pertanyaan Anda di sini"

# Ingest test (1 versi)
cd ~/apps/nora/backend && source .venv/bin/activate && \
python -m nora.ingest.run --limit 1

# Ingest resume (skip yang sudah ada)
cd ~/apps/nora/backend && source .venv/bin/activate && \
python -m nora.ingest.run --skip-existing
```

Variabel Environment Kunci

Variabel	Default	Keterangan
<code>NORA_EMBED_BACKEND</code>	<code>9router</code>	Pilihan: <code>9router</code> / <code>ollama</code> / <code>local</code>

Variabel	Default	Keterangan
NORA_ROUTER_URL	http://host.docker.internal:20128/v1	URL 9router
NORA_ROUTER_KEY	sk-local	API key 9router
NORA_GEN_MODEL	cc/claude-opus-4-8	Model generator
NORA_VERIFY_MODEL	cc/claude-sonnet-4-6	Model verifier
NORA_EMBED_MODEL	gemini/gemini-embedding-001	Model embedding (9router)
NORA_CHROMA_PATH	/app/data/chroma	Path ChromaDB dalam container
NORA_COLLECTION	ts24008	Nama collection ChromaDB
SECRET_KEY	(wajib diisi)	JWT secret — generate via <code>openssl rand -hex 32</code>

Port Akses

Service	Port Host	Keterangan
Frontend (UI)	3030	http://localhost:3030
Backend (API)	8010	http://localhost:8010
PostgreSQL	5440	localhost:5440
Produksi	443/HTTPS	https://nora.arahkarya.com (Cloudflare tunnel)

Dokumen ini disiapkan oleh Tim ArahKarya untuk keperluan operasional internal. Knowledge base: 3GPP TS 24.008 — 257 versi (R98 1999 – R18 2026). NORA Agent Layer ditenagai dual-model via 9router.

© 2026 PT Arah Karya Sinergi × NOZ

BAGIAN III

Glosarium

Definisi 60+ istilah: RAG & AI, komponen NORA, telco/3GPP, infrastruktur & operasional.

Glosarium NORA — Network Oracle for Reliable Answers

Dokumen	Glosarium NORA v1.0
Produk	NORA — Network Oracle for Reliable Answers
Kolaborasi	NOZ x PT Arah Karya Sinergi
Disiapkan oleh	Tim ArahKarya
Tanggal	Juni 2026
Status	Final

Pendahuluan

Dokumen ini merupakan glosarium resmi istilah-istilah teknis yang digunakan dalam proyek **NORA (Network Oracle for Reliable Answers)** — platform AI Research Engine berbasis RAG untuk standar telekomunikasi, dikembangkan oleh **NOZ x PT Arah Karya Sinergi**.

Istilah dikelompokkan dalam empat kategori:

- **A** — RAG & AI: konsep dasar sistem retrieval dan kecerdasan buatan yang mendasari NORA.
- **B** — Komponen NORA: komponen arsitektur dan modul internal NORA.
- **C** — Telco / 3GPP: terminologi domain standar telekomunikasi yang menjadi knowledge base NORA.
- **D** — Infrastruktur & Ops: teknologi, alat, dan konfigurasi lingkungan deploy NORA.

Setiap penjelasan ditulis ringkas dan akurat dalam konteks penggunaan NORA. Urutan istilah dalam tiap kategori mengikuti abjad.

A — RAG & AI

Istilah	Penjelasan
Anti-halusinasi / Grounding	Prinsip NORA bahwa setiap jawaban <i>wajib</i> bersumber dari potongan spec resmi yang diambil via retrieval, bukan dari ingatan parametrik model. Jawaban yang tidak didukung konteks ditolak atau diberi flag LOW CONFIDENCE.
Chunk / Chunking	Proses memecah dokumen spec panjang menjadi potongan-potongan teks (chunk) yang lebih kecil sebelum diindeks ke vector store. Di NORA, chunking dilakukan berbasis nomor section spec (section-aware chunking), bukan pemotongan ukuran tetap.
Confidence Score	Nilai numerik 0-1 yang merepresentasikan seberapa yakin NORA terhadap kebenaran suatu jawaban. Dihitung berdasarkan verdict verifier, rata-rata skor similarity chunk terambil, dan ada/tidaknya klaim tanpa dukungan sumber. Nilai < 0,7 memicu flag "LOW CONFIDENCE".
Context Window	

Istilah	Penjelasan
	Batas maksimum token yang dapat diproses sekaligus oleh sebuah LLM dalam satu inferensi. Di NORA, konteks diisi dengan chunk hasil retrieval + query user; ukurannya menentukan berapa banyak chunk bisa disertakan sebelum dikirim ke Generator.
Cosine Similarity	Metrik kemiripan antara dua vektor embedding — dihitung dari kosinus sudut antarkeduanya (rentang 0-1, makin tinggi makin mirip). NORA memakai cosine similarity untuk menentukan chunk mana yang paling relevan terhadap query user saat retrieval.
Dimensi (dim) Embedding	Jumlah dimensi numerik dalam satu vektor embedding. Semakin tinggi dimensi, semakin banyak informasi semantik yang dapat ditangkap, namun makin besar pula penggunaan memori. Contoh di NORA: Gemini embedding menghasilkan vektor berdimensi 3072, fastembed 384.
Dual-Model	Arsitektur NORA yang memakai dua model LLM berbeda secara berurutan: Generator (Claude Opus) menghasilkan jawaban awal, lalu Verifier (Claude Sonnet) memeriksa jawaban tersebut terhadap konteks sebelum jawaban dikirimkan ke user. Tujuannya: mengurangi halusinasi via self-validation.
Embedding	Representasi teks (query atau chunk) sebagai vektor numerik berdimensi tinggi yang menangkap makna semantiknya. Di NORA, embedding digunakan untuk mengubah query user dan isi spec ke dalam ruang vektor yang sama sehingga kemiripan semantik dapat dihitung secara matematis.
Generator	Peran model LLM pertama dalam pipeline dual-model NORA. Generator (Claude Opus via 9router) menerima prompt yang berisi sistem instruksi, chunk konteks dari hasil retrieval, dan query user, lalu menghasilkan jawaban teknis awal.
Inference	Proses menjalankan model LLM untuk menghasilkan output (jawaban atau verdict) dari input yang diberikan. Setiap query NORA memerlukan minimal dua inference: satu di Generator dan satu di Verifier.
LLM (Large Language Model)	Model bahasa besar yang dilatih pada data teks skala masif dan mampu menghasilkan, merangkum, serta menalar teks secara natural. Di NORA, LLM digunakan sebagai Generator (Opus) dan Verifier (Sonnet); LLM tidak menjawab dari ingatan, melainkan dari konteks chunk yang disuplai via RAG.
Loop Verifikasi	Mekanisme iteratif di NORA Agent Layer: jika Verifier menilai jawaban Generator sebagai INVALID, sistem meregenerasi jawaban dengan prompt termodifikasi (maksimum 2 iterasi ulang). Bertujuan meningkatkan kualitas jawaban secara otomatis tanpa intervensi user.
Prompt Orchestration	Proses penyusunan dan pengelolaan prompt secara terstruktur oleh NORA Agent Layer, mencakup sistem instruksi, injeksi chunk konteks, dan query user. Prompt

Istilah	Penjelasan
	Generator dan prompt Verifier dikonstruksi berbeda sesuai perannya.
RAG (Retrieval-Augmented Generation)	Arsitektur AI yang menggabungkan pencarian (retrieval) dokumen relevan dari knowledge base dengan kemampuan generasi teks LLM. NORA memakai RAG agar jawaban selalu bersumber dari spec resmi (tergrounded), bukan dari memori parametrik model yang rawan halusinasi.
Section-Aware Chunking	Strategi chunking yang digunakan NORA: dokumen spec dipecah berdasarkan nomor section (mis. 4.7.3 GPRS Attach Procedure), bukan berdasarkan jumlah kata/token tetap. Menghasilkan chunk yang bermakna secara struktural dan mempermudah pelacakan sumber ke section asli spec.
Top-k Retrieval	Pengambilan sejumlah k chunk teratas (paling mirip secara semantik) dari vector store sebagai konteks untuk Generator. Di NORA, default k = 5; chunk dipilih berdasarkan cosine similarity tertinggi terhadap embedding query user dalam collection Topik yang aktif.
Vector Store / Vector Database	Basis data yang dirancang untuk menyimpan dan mencari vektor embedding secara efisien menggunakan operasi kemiripan (similarity search). Di NORA, vector store menyimpan embedding semua chunk spec; query dicocokkan ke sini saat retrieval. Implementasi: ChromaDB (demo) / Qdrant (produksi).
Vektor / Vector	Array numerik berdimensi tinggi hasil proses embedding yang merepresentasikan makna semantik suatu teks. Dalam konteks NORA, setiap chunk spec dan setiap query user diubah menjadi vektor sehingga perbandingan kemiripan makna dapat dilakukan secara komputasional.
Verifier	Peran model LLM kedua dalam pipeline dual-model NORA. Verifier (Claude Sonnet via 9router) menerima jawaban Generator beserta chunk konteks, lalu menilai apakah jawaban tersebut didukung penuh oleh konteks. Verdict: VALID / PARTIAL / INVALID.

B — Komponen NORA

Istilah	Penjelasan
9router	Gateway model yang digunakan NORA untuk merutekan permintaan ke model cloud: Claude Opus (Generator), Claude Sonnet (Verifier), dan Gemini embedding. Bertindak sebagai lapisan abstraksi antara NORA Agent Layer dan provider LLM eksternal, sehingga engine bisa diganti via konfigurasi.
ChromaDB	Vector store embedded yang digunakan NORA pada mode demo/pengembangan (termasuk di RPi5). ChromaDB berjalan dalam proses yang sama dengan backend, ringan di memori, dan cocok untuk volume data

Istilah	Penjelasan
	medium. Untuk produksi skala besar, ChromaDB dapat digantikan Qdrant tanpa mengubah arsitektur.
Claude Opus (Generator)	Model cc/claude-opus-4-8 dari Anthropic, diakses via 9router, berperan sebagai Generator dalam pipeline dual-model NORA. Opus menghasilkan jawaban teknis awal berdasarkan prompt + chunk konteks; dipilih karena kemampuan reasoning mendalam pada domain teknis kompleks seperti spec 3GPP.
Claude Sonnet (Verifier)	Model cc/claude-sonnet-4-6 dari Anthropic, diakses via 9router, berperan sebagai Verifier dalam pipeline dual-model NORA. Sonnet memeriksa jawaban Opus terhadap konteks dan mengeluarkan verdict VALID/PARTIAL/INVALID; dipilih karena lebih hemat biaya dari Opus untuk tugas verifikasi.
Collection	Unit penyimpanan terisolasi dalam ChromaDB (atau Qdrant) yang mengelompokkan semua chunk embedding milik satu Topik. Setiap Topik NORA memiliki collection sendiri (mis. ts24008 untuk 3GPP TS 24.008) sehingga retrieval tidak bocor lintas-Topik.
fastembed	Library embedding lokal ringan (Python) yang menghasilkan vektor berdimensi 384. Digunakan NORA sebagai alternatif embedding mode lokal (NORA_EMBED_BACKEND=local) ketika 9router tidak tersedia. Tidak membutuhkan GPU, cocok untuk RPi5; trade-off: dimensi lebih rendah dari Gemini.
Gemini Embedding (gemini-embedding-001 dim3072)	Model embedding dari Google, diakses via 9router, menghasilkan vektor berdimensi 3072. Ini adalah embedding default NORA karena menghasilkan representasi semantik kaya tanpa membebani RAM RPi5 (komputasi di cloud). Model ID: gemini/gemini-embedding-001 .
Ingest	Proses memasukkan dokumen spec (file .txt per rilis) ke dalam pipeline NORA: chunking per-section → embedding tiap chunk → penyimpanan ke vector store (ChromaDB) beserta metadata (spec, versi, section). Ingest dilakukan sekali per Topik/rilis; re-ingest dipicu saat ada update spec.
Knowledge Base (KB)	Kumpulan dokumen spec resmi yang telah diproses dan diindeks untuk satu Topik NORA. Untuk Topik 3GPP TS 24.008, KB terdiri dari 257 rilis versi (.txt hasil konversi dari .doc 3GPP) yang tersimpan di direktori data/3gpp/24008/txt .
Multi-Topik	Kemampuan NORA melayani beberapa knowledge base standar telco yang berbeda secara bersamaan. Setiap Topik memiliki collection dan KB sendiri namun berbagi pipeline RAG yang sama. User memilih Topik sebelum bertanya; admin bisa menambah Topik baru tanpa mengubah kode engine.
NORA Agent Layer	Inti orkestrasi mandiri NORA — kode Python murni internal NORA (bukan runtime AI generik pihak ketiga) yang menjalankan loop retrieve → generate (Opus) → verify (Sonnet) → validate → loop ulang jika INVALID.

Istilah	Penjelasan
	Bersifat stateless-per-request, multi-tenant: state (memory, history) disimpan per-user di PostgreSQL, sehingga dapat di-load-balance dan di-scale horizontal.
Ollama	Runtime model LLM lokal yang memungkinkan NORA beroperasi sepenuhnya tanpa panggilan ke layanan cloud. Digunakan sebagai alternatif swap dari 9router (<code>NORA_ENGINE=ollama</code>) untuk mode privacy-first. Model yang didukung: Llama3.1, Qwen, beserta embedding lokal (nomic-embed-text, mxbai-embed-large).
PostgreSQL	Basis data relasional yang menyimpan seluruh state persisten NORA: data user dan autentikasi, memory percakapan per-user, history query, log interaksi, serta feedback. Penggunaan Postgres memungkinkan NORA Agent Layer tetap stateless-per-request dan aman di-replika.
Query Pipeline	Rangkaian langkah yang dieksekusi NORA Agent Layer untuk setiap query user: (1) embedding query, (2) retrieval chunk dari collection Topik aktif, (3) konstruksi prompt, (4) generate jawaban (Opus), (5) verify (Sonnet), (6) validasi + hitung confidence, (7) kirim output.
Re-ingest	Proses menjalankan ulang ingest untuk spec atau rilis yang telah diperbarui. Digunakan saat ada versi spec baru yang perlu diindeks ke ChromaDB. Rilis yang sudah terindeks dapat di-skip (delta ingestion) untuk efisiensi.
Topik	Unit knowledge base independen dalam NORA, merepresentasikan satu domain/spec standar telco (mis. 3GPP TS 24.008). Setiap Topik memiliki atribut: <code>id</code> , <code>nama</code> , <code>deskripsi</code> , <code>collection</code> (ChromaDB), <code>kb_dir</code> (lokasi file .txt), dan <code>status</code> (live/indexing/planned). Topik pertama yang live: 3GPP TS 24.008 .

C — Telco / 3GPP

Istilah	Penjelasan
3GPP (3rd Generation Partnership Project)	Konsorsium standar internasional yang menghasilkan spesifikasi teknis untuk jaringan telekomunikasi seluler generasi 2G hingga 5G dan seterusnya. NORA menggunakan dokumen spec 3GPP (khususnya seri TS 24.xxx) sebagai sumber knowledge base utamanya.
CC (Call Control)	Prosedur Layer 3 NAS yang mengelola pembangunan, pengelolaan, dan pemutusan panggilan suara antara Mobile Station dan jaringan core. CC terdefinisi dalam TS 24.008 dan menjadi salah satu domain utama knowledge base NORA Topik #1.
GMM (GPRS Mobility Management)	Sublayer NAS yang mengelola mobilitas dan registrasi perangkat yang menggunakan layanan data paket GPRS: prosedur Attach, Detach, dan Routing Area Update. GMM tercakup dalam TS 24.008 dan terindeks dalam knowledge base NORA.

Istilah	Penjelasan
GPRS Attach	Prosedur registrasi yang dilakukan Mobile Station ke jaringan GPRS untuk mengaktifkan layanan data paket, termasuk pertukaran pesan (mis. Attach Request, Attach Accept/Reject) antara MS dan SGSN. Prosedur ini terdefinisi di TS 24.008 §4.7.3 dan merupakan contoh query utama NORA.
Knowledge Base Telco	Dalam konteks NORA: koleksi dokumen spec telekomunikasi resmi (3GPP, ITU-T, IEEE, GSMA) yang telah diproses, di-chunk, di-embed, dan diindeks ke vector store sebagai sumber jawaban. Setiap knowledge base terikat pada satu Topik.
Layer 3 / L3	Lapisan ketiga dalam model protokol jaringan — di konteks 3GPP, L3 merujuk pada layer protokol NAS (Non-Access Stratum) antara Mobile Station dan jaringan core, mencakup Mobility Management, Call Control, dan Session Management. TS 24.008 mendefinisikan protokol L3 ini.
MM (Mobility Management)	Sublayer NAS yang mengelola prosedur mobilitas dan registrasi perangkat 2G/3G di jaringan Circuit-Switched: Location Updating, IMSI Attach/Detach, dan Identification. MM terdefinisi di TS 24.008 dan terindeks dalam knowledge base NORA.
Mobile Radio Interface	Antarmuka udara antara Mobile Station (handset) dan jaringan akses radio (BTS/NodeB). Istilah ini muncul dalam nama lengkap TS 24.008: " <i>Mobile radio interface layer 3 specification; Core network protocols</i> ", menandai scope protokol yang dicakup spec tersebut.
NAS (Non-Access Stratum)	Lapisan protokol dalam arsitektur 3GPP yang berjalan di atas lapisan akses radio (AS/RAN), menghubungkan Mobile Station langsung ke jaringan core (MSC/SGSN/MME). Protokol NAS mencakup Mobility Management, Session Management, dan Call Control — semuanya terdefinisi di TS 24.008.
Release (R98 - R18)	Versi tahunan/periodik dari standar 3GPP yang mengelompokkan sekumpulan fitur dan perbaikan. NORA mengindeks 257 rilis TS 24.008 mulai Release 98 (R98, v3.0.0, tahun 1999) hingga Release 18 (R18, seri j, 2026), memungkinkan query berbasis versi spesifik.
Spec / Specification	Dokumen teknis resmi 3GPP yang mendefinisikan protokol, prosedur, format pesan, dan parameter jaringan telekomunikasi. Spec adalah sumber primer knowledge base NORA; setiap jawaban NORA harus dapat ditelusuri ke section spesifik sebuah spec.
TS 24.008	Technical Specification 3GPP: " <i>Mobile radio interface layer 3 specification; Core network protocols; Stage 3</i> " — mendefinisikan seluruh protokol L3 NAS (MM, GMM, CC, SM) antara MS dan jaringan core. Ini adalah Topik #1 NORA (reference implementation), dengan 257 rilis yang terindeks.

D — Infrastruktur & Ops

Istilah	Penjelasan
bcrypt	Algoritma hashing password adaptif yang digunakan NORA untuk menyimpan password user secara aman di PostgreSQL. bcrypt secara sengaja lambat (cost factor dapat dikonfigurasi) sehingga tahan terhadap serangan brute-force.
Cloudflare Tunnel	Layanan tunneling terenkripsi dari Cloudflare yang memungkinkan NORA di RPi5 (jaringan lokal/private) dapat diakses secara aman dari internet tanpa membuka port inbound di firewall. Tidak memerlukan IP publik statis; koneksi keluar dari RPi5 ke Cloudflare edge.
Container	Unit deployment terisolasi yang mengemas aplikasi beserta dependensinya (kode, runtime, library, konfigurasi). NORA menggunakan container Docker untuk setiap komponen (backend FastAPI, ChromaDB, frontend Next.js) agar dapat di-deploy dan dipindah lintas server secara konsisten.
CORS (Cross-Origin Resource Sharing)	Mekanisme keamanan browser yang mengontrol permintaan HTTP antara origin yang berbeda (mis. frontend Next.js ke backend FastAPI). NORA mengkonfigurasi CORS agar hanya origin yang diizinkan dapat mengakses API backend.
CSP / HSTS	CSP (Content Security Policy) : header HTTP yang membatasi sumber konten (script, style, gambar) yang boleh dimuat browser, mencegah XSS. HSTS (HTTP Strict Transport Security) : header yang memaksa browser selalu menggunakan HTTPS. Keduanya dikonfigurasi di NORA untuk hardening keamanan web.
Dimensi Mismatch	Error yang terjadi ketika embedding yang tersimpan di vector store memiliki dimensi berbeda dari embedding query yang dikirim saat retrieval (mis. 3072 vs 384). Di NORA, mismatch ini bisa terjadi jika backend embedding diganti tanpa re-ingest ulang seluruh collection.
Docker / Docker Compose	Docker : platform containerisasi yang mengemas aplikasi dan dependensinya dalam container portabel. Docker Compose : alat orkestrasi multi-container yang mendefinisikan dan menjalankan seluruh stack NORA (backend + ChromaDB + frontend) via satu file <code>docker-compose.yml</code> .
earlyoom	Daemon Linux yang memonitor penggunaan memori secara aktif dan mematikan proses paling boros RAM sebelum kernel OOM killer beraksi. Diinstal di RPi5 untuk menjaga stabilitas NORA saat menjalankan proses embedding atau LLM lokal (Ollama) yang menekan memori.
httpOnly Cookie	Cookie dengan flag <code>HttpOnly</code> yang tidak dapat diakses JavaScript di browser — hanya dikirim melalui header HTTP. NORA menggunakan httpOnly cookie untuk

Istilah	Penjelasan
	menyimpan session token/JWT agar terlindungi dari serangan XSS yang mencoba mencuri token autentikasi.
JWT (JSON Web Token)	Token terenkripsi berbasis standar RFC 7519 yang digunakan NORA untuk autentikasi stateless: setelah login, server menerbitkan JWT yang disimpan di httpOnly cookie; setiap request selanjutnya menyertakan token ini untuk verifikasi identitas user.
mmap	Mekanisme memory-mapped file di Linux yang memungkinkan file dibaca langsung ke virtual address space proses tanpa menyalin ke heap. Relevan di RPi5 saat ChromaDB atau Ollama memuat file model/index besar: mmap mengurangi penggunaan RAM aktif karena halaman dimuat on-demand.
OOM (Out of Memory)	Kondisi saat sistem Linux kehabisan RAM yang tersedia dan kernel terpaksa mematikan proses (OOM killer). Di RPi5 8GB, risiko OOM meningkat saat menjalankan Ollama (model lokal besar) bersamaan dengan ChromaDB dan backend. Mitigasi: earlyoom + swap.
PWA (Progressive Web App)	Teknologi web yang memungkinkan aplikasi NORA diinstall di perangkat user (desktop/mobile) dan bekerja seperti aplikasi native, termasuk dukungan offline terbatas via Service Worker. Frontend Next.js NORA dirancang dengan kapabilitas PWA.
RPi5 (Raspberry Pi 5)	Single-board computer Raspberry Pi generasi 5 (8GB RAM) yang menjadi target hardware deploy NORA untuk demo dan pengembangan. RPi5 menjalankan seluruh stack NORA via Docker Compose + Cloudflare Tunnel; arsitektur ARM64 dengan keterbatasan memori menjadi pertimbangan desain (pemilihan ChromaDB, Gemini embedding cloud, earlyoom).
Service Worker	Script JavaScript yang berjalan di background browser, terpisah dari halaman web, dan memungkinkan fitur PWA seperti caching aset dan dukungan offline. Digunakan frontend NORA untuk meningkatkan performa load dan memungkinkan instalasi sebagai PWA.
Swap / Swappiness	Swap: ruang disk yang digunakan kernel Linux sebagai "RAM darurat" ketika RAM fisik penuh — memperlambat sistem tapi mencegah OOM. Swappiness: parameter kernel (0–100) yang mengontrol agresivitas penggunaan swap. Di RPi5, swap dikonfigurasi + swappiness diturunkan untuk menyeimbangkan stabilitas dan performa NORA.
venv (Virtual Environment)	Lingkungan Python terisolasi yang memisahkan dependensi proyek NORA dari paket sistem Python. Digunakan dalam pengembangan dan deployment non-Docker untuk mencegah konflik versi library (mis. FastAPI, ChromaDB, fastembed).